Project Report

Yulin Wu, #21130792

November 16, 2024

1 Introduction

The vertex cover problem is an NP-hard problem that tries to find a subset of the vertices in a graph, such that every edge in the graph is incident on some vertex in the subset. This subset is called a vertex cover. The objective of minimum vertex cover problem is to find the minimum size of a vertex cover.

In the previous assignment of the course ECE 650 in 2024 Fall, an approach that encodes the problem into conjunctive normal form (CNF) and use a SAT solver to compute the minimum vertex cover. This project implements two approximation method to compute the minimum vertex cover and compares the measure of running time and approximation ratio of different methods based on experiments of different graphs. The major find of the result shows that the CNF methods runs really slow when the graph has bigger size and approximation method can yield great approximation ratio. This gives us an idea that sometimes we can use approximation algorithm to trade time efficiency with precision. In the future development, some heuristic approximation methods or better CNF encoding can be use to improve the performance of solving this problem.

This project report is a summary and analysis of the course project of ECE 650 in 2024 Fall. In the following sections, the report first discusses the program structure and test dataset. Then, the report shows the visualized results of running time and approximation ratio, and gives some explanation of them. Finally, the report provides a conclusion and gives some advice for future development.

2 Program Structure and Test Dataset

The program of this project consists of four threads: one main thread and three other threads that compute the minimum vertex cover problems in different approaches. The main thread deals with input graph parsing and output results, creates threads to solve the problem and controls other threads.

In addition, The project implements a data structure, Graph, to store parsed graph information, i.e., edges and vertices, and also adds functions to the data structure that compute the minimum vertex cover problems in different approaches, namely CNF-SAT-VC, APPROX-1-VC and APPROX-2-VC. After the main thread parses the graph and creates a Graph data instance, it will create three solver threads, each of which calls different solving functions in the instance and collects analytic data for visualizing the running time. The main thread will wait for a reasonable amount of time for the solver threads to process, and after the time expires, try to cancel the threads that are still running and format the vertex cover result or analytic data for output printing, based on the mode macro defined in the file.

This project chose the timeout as $1,600$ milliseconds. This timeout should not be too large as the project needs to run each graph ten times; each $|V|$ has ten graphs; and it is required to collect data for ten different $|V|$. The total run would be $1,000$ times. Besides, based on the previous development on assignment 4, the CNF-SAT-VC approach runs exponentially slower as the number of vertices increases. When the approach tries vertex cover size of 7, the running time would be no more than $1,000$ milliseconds, which is a reasonable timeout. But when trying the size of 8, the running time would be a few minutes, which is too much for the timeout. Therefore, considering other time cost of other routine such as I/O, the timeout is chosen as $1,600$ milliseconds.

The test dataset used in this project is generated by a given application "GraphGen", the implementation detail of which is not given. This application generates graphs based on the number of vertices. Based on observation, it creates random graphs with the number of edges equal to the number of vertices and also guaranteed that every vertex belongs to some edge.

The test dataset is created in accordance to the project requirement: ten graphs for each $|V|$ and $|V|$ ranges from 5 to 50 in increments of 5.

3 Result and Discussion

3.1 Running Time

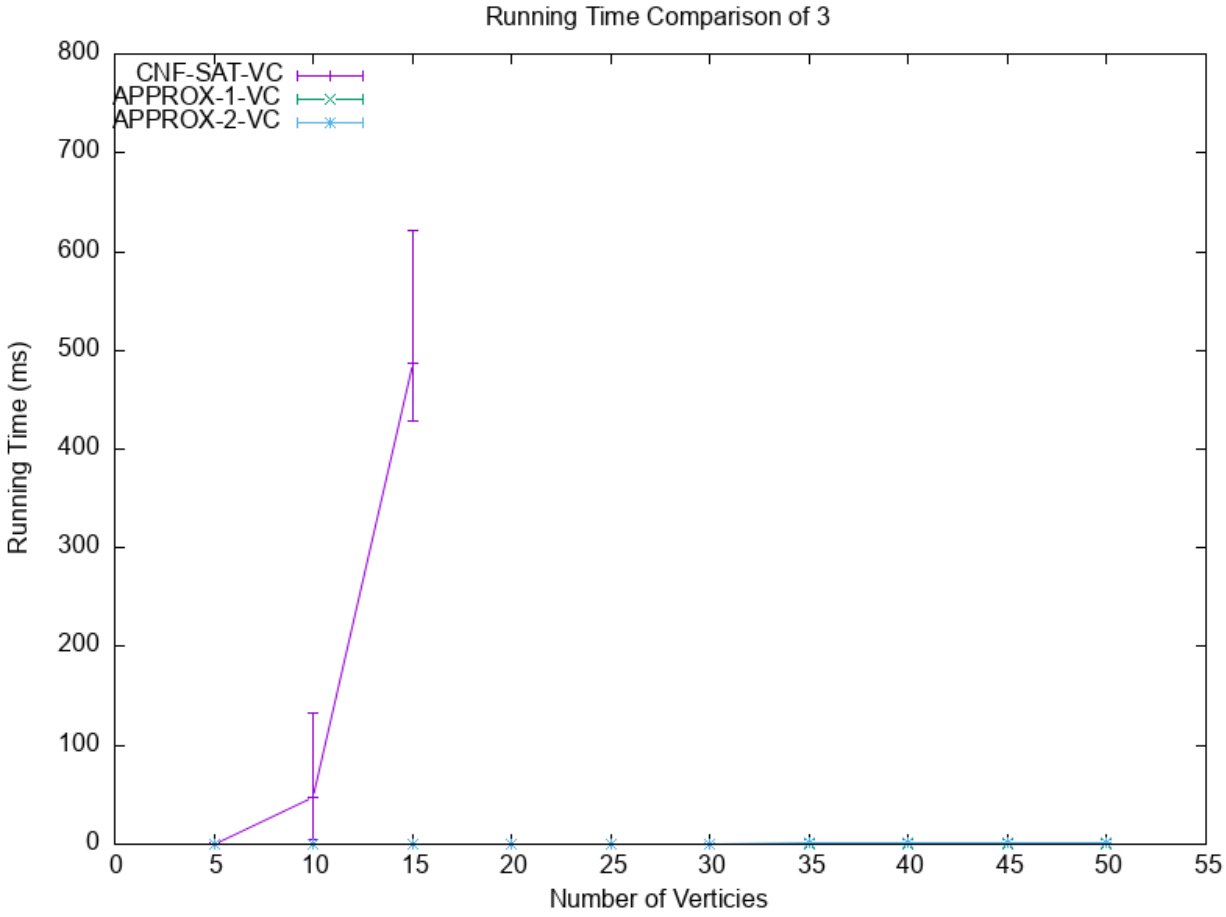

Figure 1: Running Time Comparison of CNF-SAT-VC, APPROX-1-VC and APPROX-2-VC

The running time of this project is measured by milliseconds. Figure 1 is the running time comparison of all three approaches. Figure 2 is a zoomed in running time comparison of APPROX-1-VC and APPROX-2-VC.

As shown in Figure 1, the time consumed by CNF-SAT-VC is significantly larger than the other two approaches as the number of vertices increases. The trend of the running time of CNF-SAT-VC as the number of vertices increases is exponential increase trend. The reason for this trend is that the CNF-SAT-VC approach enumerates the size of vertex cover in ascending order. Before finding the minimum vertex cover size, the approach already tried smaller size that will cause the encoded CNF unsatisfied. Since proving a CNF is unsatisfied runs at exponential time, the running time of the CNF-SAT-VC approach is also exponential. Eventually, when the number of vertices is greater and equal than 20, all graphs using CNF-SAT-VC approach result in timeout and this report omits the timeout result from the plot as timeout running time results are meaningless.

The running time standard deviation of CNF-SAT-VC is relatively large. This unstable performance might result from the allocation routine of the CNF-SAT-VC approach. That is the allocation of variables and the SAT solver, and the solver itself might use some allocation when running. These allocation operations touch the computer memory, and the running time is impacted by the operating system and the number of users sharing the computer, leading to unstable performance.

Figure 2 gives a running time comparison of APPROX-1-VC and APPROX-2-VC.

The most apparent trend in this figure is that the running time of both APPROX-1-VC and APPROX-2-VC increase linearly as the number of vertices increases. This is because both approaches have their major iteration routines linearly related to the number of edges. Since all graph dataset used for testing have the same number of edges as the number of vertices, the major iteration routines in both APPROX-1-VC and APPROX-2-VC are linearly related to the number of vertices.

It is also easy to notice that the standard deviation of APPROX-2-VC is significantly larger than APPROX-1-VC. The reason for this difference is that APPROX-2-VC utilize a random subroutine to pick edges. This random process generates true random number from `/dev/urandom` to pick edges, which requires reading from a file. File reading process is managed by the operating system and also is affected by other users on the machine, resulting in unstable performance. On the contrary, APPROX-1-VC does not contain any file reading process. Apart from only one allocation of a bookkeeping vector for counting the number of edges incident on the vertices, other operations are numeric operations, which are stabler than file reading.

Another obvious trend is that the running time of APPROX-2-VC increases faster than the running time of APPROX-1-VC. This is also resulted from the file reading process in generating random picks of edges in APPROX-2-VC. As the number of vertices increases, according to the observation of the test dataset, the number of edges equals to the number of vertices and also increases, leading to the number of random picks and the number of other numeric operations increases. Since the file reading process consume more time than other numeric operations, the running time of APPROX-2-VC increases faster than the running time of APPROX-1-VC as the number of vertices increases.

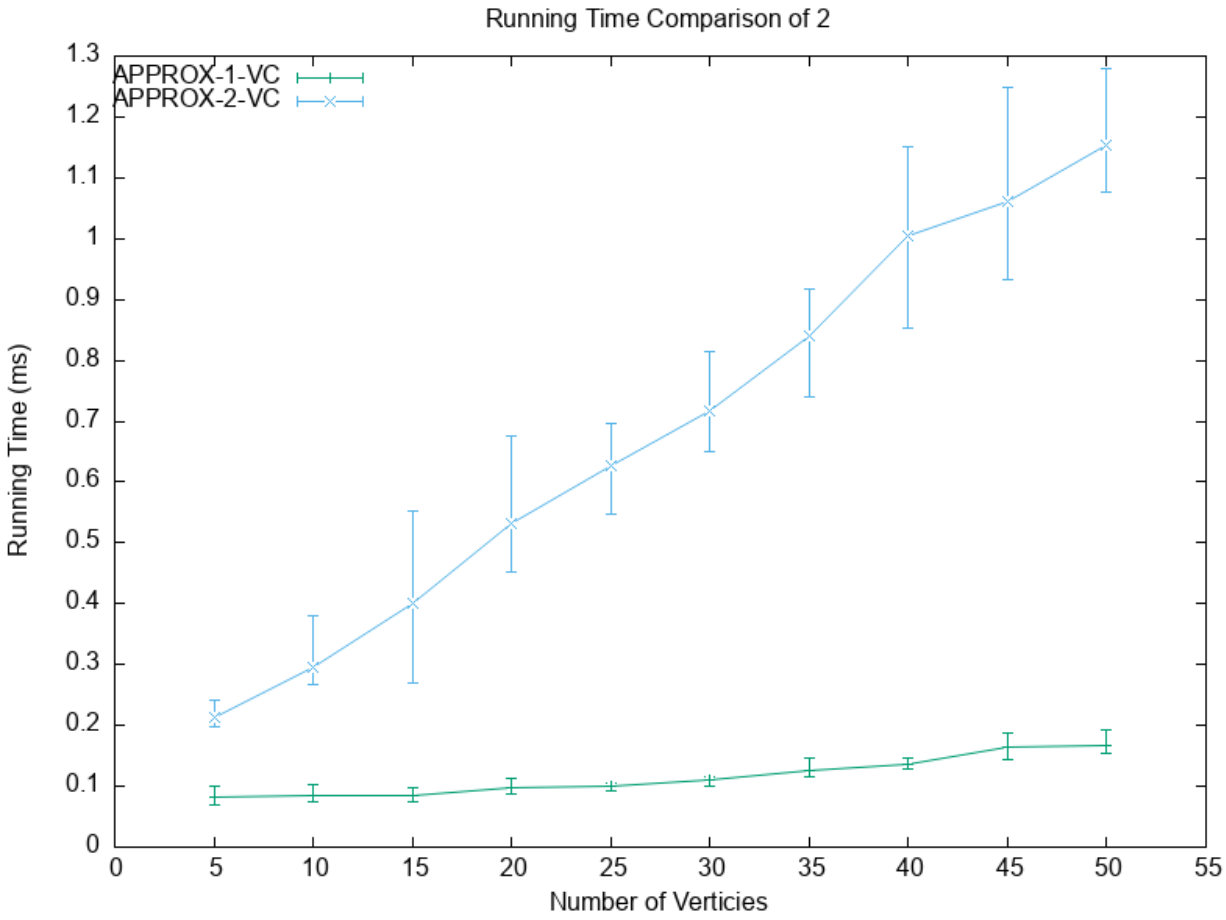

Figure 2: Running Time Comparison of APPROX-1-VC and APPROX-2-VC

3.2 Effective Ratio

Figure 3 demonstrates the effective ratio of APPROX-1-VC and APPROX-2-VC. The effective ratio is computed by the following equation:

$$\text{Effective Ratio} = \frac{\text{Size of Approximate Minimum Vertex Cover}}{\text{Size of Actual Minimum Vertex Cover}} \tag{1}$$

The size of actual minimum vertex cover in equation (1) is the result by CNF-SAT-VC. Since CNF-SAT-VC generates timeout and no minimum vertex cover is found after the number of vertices is greater than 15, the effective ratio is not plotted for the number of vertices is greater than 15.

There are some trend shown in Figure 3. Firstly, the average effective ratio of APPROX-1-VC is very close to 1. This shows that always picking the most incident vertex as part of vertex cover is a great approximation for finding the minimum vertex cover. Secondly, the standard deviation effective ratio of APPROX-2-VC is larger than APPROX-1-VC. Because of random edge picking used in APPROX-2-VC, picking the an edge with both end points in the minimum vertex cover is a bang for luck, causing this unstable performance. Finally, the average effective ratio of APPROX-2-VC is 1.5 times of that of APPROX-1-VC. This is because the expected number of vertices of an edge in the minimum vertex cover is 1.5. Let random variable X denotes the number of of vertices of an edge in the minimum vertex cover. Then, $X = 1$ or $X = 2$. Since the graph is random

generated, the probabilities $Pr\{X = 1\} = 1/2$ and $Pr\{X = 2\} = 1/2$. Therefore, the expected number of vertices of an edge in the minimum vertex cover is $E[X] = 1 \times 1/2 + 2 \times 1/2 = 3/2 = 1.5$.

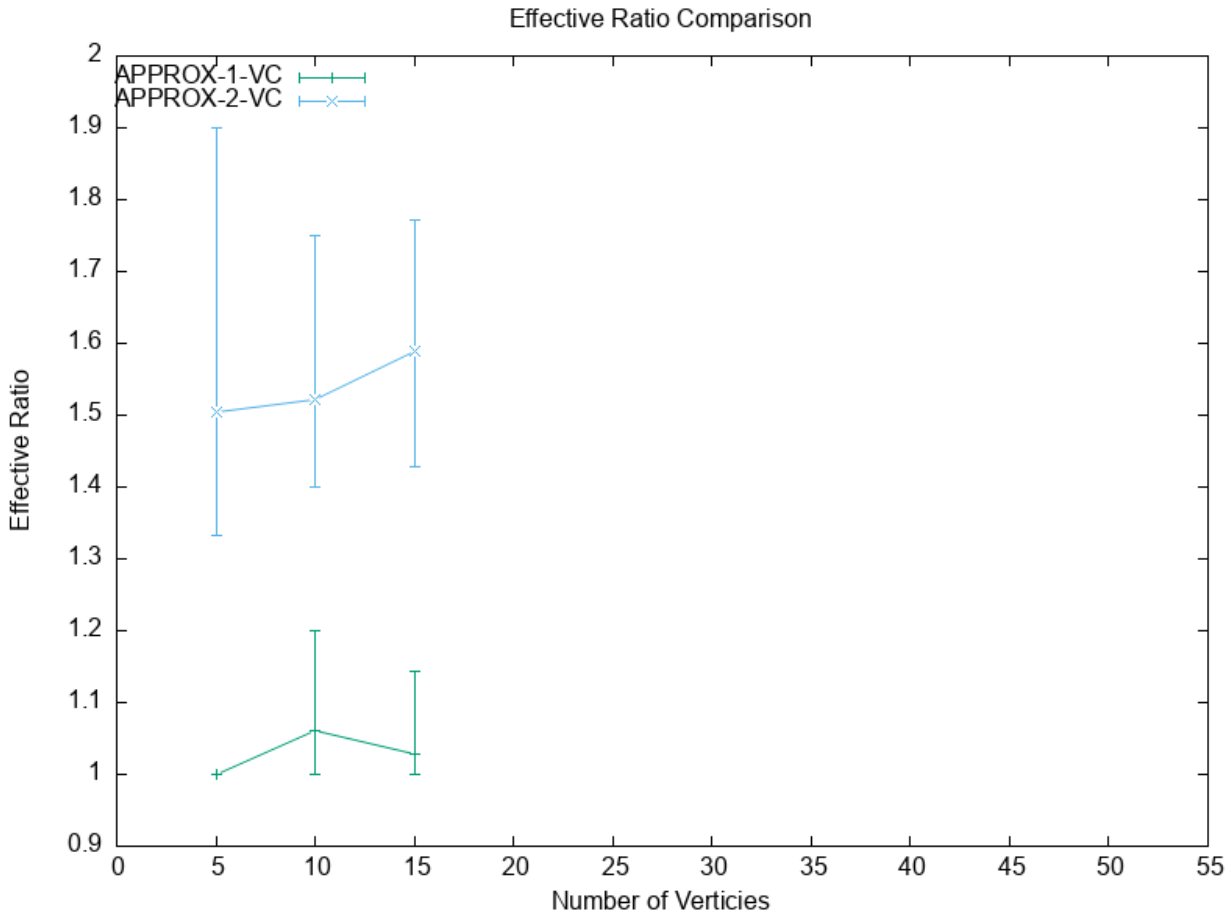

Figure 3: Effective Ratio Comparison of APPROX-1-VC and APPROX-2-VC

4 Conclusion

In the analysis of running time, CNF-SAT-VC runs exponential slower than APPROX-1-VC and APPROX-2-VC. The approximation methods runs at a linear time. In the analysis of effective ratio, APPROX-1-VC yields a really good approximation result compared to APPROX-2-VC. Although CNF-SAT-VC generates guaranteed minimum vertex cover, considering its awful running time, using the approximation approach APPROX-1-VC can be a better idea.

In the future development, either reducing generated clauses by the CNF encoding of the minimum vertex cover problem to improve the running time of CNF-SAT-VC, or improve the approximation methods like adding some heuristic picking process can become a great research direction.